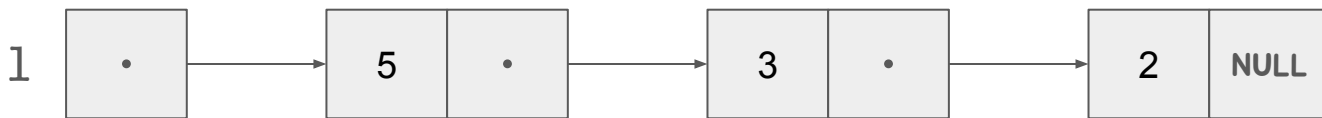


CRIS workshop Day3

N번째 소수찾기 프로그램 만들기 · 스펙적기 · 증명하기

오늘 증명할 프로그램은?

- 연결 리스트 (Singly Linked List)를 활용해 N번째 소수찾기 프로그램
- 연결 리스트란?



길이 3짜리 (소수를 담은) 연결리스트 (desc. order)

```
1 struct llist_node {  
2     int value;  
3     struct llist_node *next;  
4 };
```

```
Module LList.  
  Definition init := ...  
  Definition get (n: nat) := ...  
  Definition push_front (l: loc, v: val) := ...  
End LList.
```

오늘 증명할 프로그램은?

```
1 int get_prime(void) {
2     int n;
3     struct llist *list_loc = LList.new();
4     int candidate = 2, found = 0;
5
6     scanf("%d", &n); // n에 사용자 입력 받기
7
8     while(1) {
9         int composite = 0; // candidate이 소수인지 합성수인지 확인하는 flag
10        int index = 0;
11
12        // 현재까지 발견된 소수로 나눠보기
13        while (index != found) {
14            int divisor;
15            divisor = LList.get(index);
16            if (candidate % divisor == 0) { composite = 1; break; }
17            index++;
18        }
19
20        if (!composite) { // 소수였다면 linked list에 소수 삽입
21            LList.push_front(list, candidate);
22            // 만약 지금까지 총 n개의 소수를 찾았다면 바로 그 소수가 정답!
23            if (found == n) return candidate;
24            found++;
25        }
26        candidates++;
27    }
28 }
```

...(소수 2, 3, 5가 linked list에 담겨있는 상태)...

- 현재 : n=3; candidate = 7; found = 2



오늘 증명할 프로그램은?

```
1 int get_prime(void) {
2     int n;
3     struct llist *list_loc = LList.new();
4     int candidate = 2, found = 0;
5
6     scanf("%d", &n); // n에 사용자 입력 받기
7
8     while(1) {
9         int composite = 0; // candidate이 소수인지 합성수인지 확인하는 flag
10        int index = 0;
11
12        // 현재까지 발견된 소수로 나눠보기
13        while (index != found) {
14            int divisor;
15            divisor = LList.get(index);
16            if (candidate % divisor == 0) { composite = 1; break; }
17            index++;
18        }
19
20        if (!composite) { // 소수였다면 linked list에 소수 삽입
21            LList.push_front(list, candidate);
22            // 만약 지금까지 총 n개의 소수를 찾았다면 바로 그 소수가 정답!
23            if (found == n) return candidate;
24            found++;
25        }
26        candidates++;
27    }
28 }
```

...(소수 2, 3, 5가 linked list에 담겨있는 상태)...

- 현재 : n=3; candidate = 7; found = 2



index = 0
divisor = 5
composite ? X

오늘 증명할 프로그램은?

```
1 int get_prime(void) {
2     int n;
3     struct llist *list_loc = LList.new();
4     int candidate = 2, found = 0;
5
6     scanf("%d", &n); // n에 사용자 입력 받기
7
8     while(1) {
9         int composite = 0; // candidate이 소수인지 합성수인지 확인하는 flag
10        int index = 0;
11
12        // 현재까지 발견된 소수로 나눠보기
13        while (index != found) {
14            int divisor;
15            divisor = LList.get(index);
16            if (candidate % divisor == 0) { composite = 1; break; }
17            index++;
18        }
19
20        if (!composite) { // 소수였다면 linked list에 소수 삽입
21            LList.push_front(list, candidate);
22            // 만약 지금까지 총 n개의 소수를 찾았다면 바로 그 소수가 정답!
23            if (found == n) return candidate;
24            found++;
25        }
26        candidates++;
27    }
28 }
```

...(소수 2, 3, 5가 linked list에 담겨있는 상태)...

- 현재 : n=3; candidate = 7; found = 2



index = 1
divisor = 3
composite ? X

오늘 증명할 프로그램은?

```
1 int get_prime(void) {
2     int n;
3     struct llist *list_loc = LList.new();
4     int candidate = 2, found = 0;
5
6     scanf("%d", &n); // n에 사용자 입력 받기
7
8     while(1) {
9         int composite = 0; // candidate이 소수인지 합성수인지 확인하는 flag
10        int index = 0;
11
12        // 현재까지 발견된 소수로 나눠보기
13        while (index != found) {
14            int divisor;
15            divisor = LList.get(index);
16            if (candidate % divisor == 0) { composite = 1; break; }
17            index++;
18        }
19
20        if (!composite) { // 소수였다면 linked list에 소수 삽입
21            LList.push_front(list, candidate);
22            // 만약 지금까지 총 n개의 소수를 찾았다면 바로 그 소수가 정답!
23            if (found == n) return candidate;
24            found++;
25        }
26        candidates++;
27    }
28 }
```

...(소수 2, 3, 5가 linked list에 담겨있는 상태)...

- 현재 : n=3; candidate = 7; found = 2



↑
index = 2
divisor = 2
composite ? X

오늘 증명할 프로그램은?

```
1 int get_prime(void) {
2     int n;
3     struct llist *list_loc = LList.new();
4     int candidate = 2, found = 0;
5
6     scanf("%d", &n); // n에 사용자 입력 받기
7
8     while(1) {
9         int composite = 0; // candidate이 소수인지 합성수인지 확인하는 flag
10        int index = 0;
11
12        // 현재까지 발견된 소수로 나눠보기
13        while (index != found) {
14            int divisor;
15            divisor = LList.get(index);
16            if (candidate % divisor == 0) { composite = 1; break; }
17            index++;
18        }
19
20        if (!composite) { // 소수였다면 linked list에 소수 삽입
21            LList.push_front(list, candidate);
22            // 만약 지금까지 총 n개의 소수를 찾았다면 바로 그 소수가 정답!
23            if (found == n) return candidate;
24            found++;
25        }
26        candidates++;
27    }
28 }
```

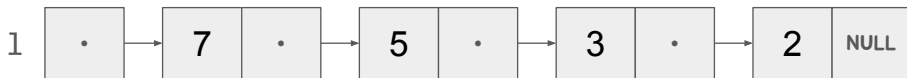
...(소수 2, 3, 5가 linked list에 담겨있는 상태)...

- 현재 : n=3; candidate = 7; found = 3



found == n ? YES!

loop 탈출



LList.push_front(1, 7)

전체 증명 구조

```
Module Prime.  
  get_prime :=  
    n <- trigger (IO nat);;  
    Ret (nth_prime n)  
End Prime.
```

(Optional)

Cancelled Prime Spec



Prime Spec
get_prime/main



LinkedList Spec
init/get/push_front



Prime Impl
get_prime/main



Memory Spec
alloc/load/store/...



LinkedList Impl
init/get/push_front



Prime Impl
get_prime/main



Memory Impl
alloc/load/store/...



LinkedList Impl
init/get/push_front

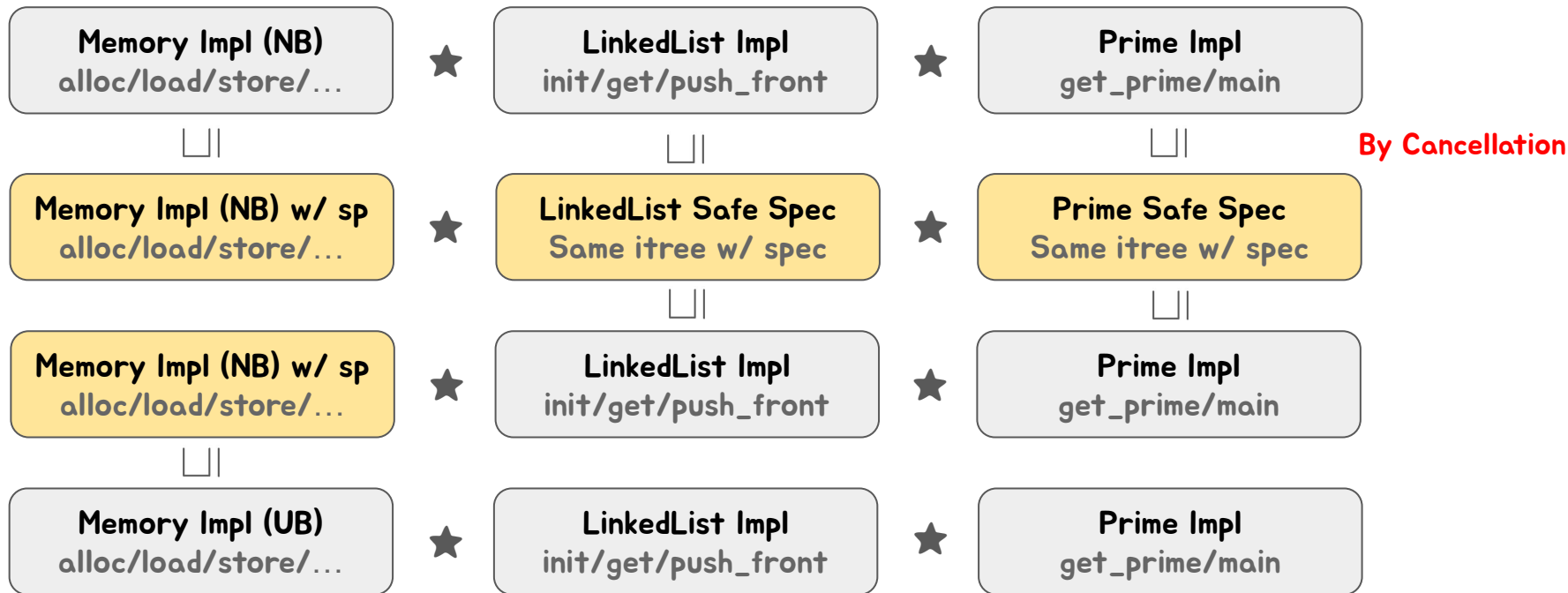


Prime Impl
get_prime/main

N번째 소수찾기 프로그램 만들기 · 스펙적기 · 증명하기

실습 시작!

[Appendix A] Prime Memory Safety



[Appendix B] Prime Example w/ Verified Compiler

Reference :

Archmage and CompCertCast: End-to-End Verification Supporting Integer-Pointer Casting (Kim et al. POPL'25)

<https://github.com/snu-sf/ClightPlus/tree/2026summer-workshop>

